



A GitHub profile that generates itself

A self-typing ASCII portrait and live contribution stats, drawn by your own repository. No third-party widgets, no services that can rate-limit or go dark, and every pixel of it under your control.

Live example: github.com/andriidrok1 · all the code: [andriidrok1/andriidrok1](https://github.com/andriidrok1/andriidrok1)

[Why not just use the usual cards](#)

[What GitHub actually allows](#)

[Setup](#)

[Part 1 — the portrait](#)

[The photo decides everything](#)

[The pipeline, and why each stage exists](#)

[The typing animation](#)

[The trap nobody mentions](#)

[Part 2 — stats your own repo draws](#)

[Pick the right chart type](#)

[Two determinism traps](#)

[The workflow](#)

[Part 3 — making the text look deliberate](#)

[Part 4 — inlining the font](#)

[Gotchas, collected](#)

[Credits](#)

Why not just use the usual cards

Most profile READMEs pull their graphics from someone else's server — stats cards, streak cards, activity graphs, the contribution snake. Two things go wrong with that.

They break. While building this, `github-readme-stats.vercel.app` returned 503 on every request for hours, and its replacement started answering `ERROR!!! Cards are temporarily rate limited`. Your profile is the one page where a broken image is most expensive.

You can't design them. You get somebody's theme list. If you want the page to look like one thing rather than five, you have to draw it yourself.

⚡ Everything below is generated inside your own repository by a scheduled action. The whole page makes **zero** third-party requests.

What GitHub actually allows

This is the part no guide writes down, and it dictates every decision that follows. Tested by posting markdown to GitHub's own rendering API (`POST /markdown`) and reading back what survived.

```
STRIPPED <style> blocks style=" " attributes class=" " inline <svg> <font> <small> <big>
KEPT <sub> <sup> <kbd> <samp> <blockquote> <details> <hr> <picture> align=" " width=" "
on <img> and <td>
```

Three consequences worth internalising:

1. **You cannot change the font of README text.** Not with CSS, not with a `font` tag, not with inline SVG. Your only choice is GitHub's sans or its monospace, via backticks or a `code` / `samp` tag.

2. **Motion must live inside the SVG.** Scripts are stripped, so animation has to be SMIL — `animate` and `set` elements — inside the file. GitHub does run those.
3. **Anything you want in your own typeface has to be an image.**

Setup

```
# the repo has to be named exactly your GitHub username
gh repo create <your-username> -public --clone
cd <your-username>
pip install pillow numpy opencv-python-headless rembg onnxruntime
```

The first run downloads a ~176 MB background-removal model. Once, then cached.

Part 1 — the portrait

Credit where it's due: the portrait pipeline comes from the ASCII Portrait README Guide, at burly-handstand-0dc.notion.site/ASCII-Portrait-README-Guide-3a3e3f86338481f0b545ec8120bbf604.

Everything after Part 1 is new, and Part 1 has a couple of fixes.

The photo decides everything

📷 No amount of parameter tuning rescues a bad input. ASCII draws with **shadow**, not detail — it has about 13 brightness levels to work with.

- **Side light.** A window at roughly 45°, everything else off. One side of the face lit, the other in shadow. Flat frontal light gives you a uniform mid-tone and the face renders as a hole.
- **Fill the frame.** Crop tight: chin to just above the hair. At 90 columns, a face occupying 30% of the frame gets about 30 characters across, and eyes won't resolve.
- **High resolution.** A 320px headshot failed here repeatedly — thin features like glasses frames get averaged away on downscale. A 1200px+ crop worked.
- **Plain background**, and don't wear black against a dark wall.
- **Slight angle**, not dead-on. Gives the nose and jaw a shadow edge.

The pipeline, and why each stage exists

Stage	Why it's there
<code>rembg</code> cut-out	everything outside the subject is forced to white, which maps to the blank end of the ramp. Skip it and the background fills with <code>@</code> and drowns the portrait
bilateral filter	smooths skin while keeping edges
CLAHE, clip ≈ 3.0	local contrast per tile. Global autocontrast leaves a flatly-lit face as one tone
darkening curve <code>(v/255)^1.7</code>	the fix. With the original guide's defaults the face came out washed out and featureless. This curve is what makes glasses, brows and lips survive
map to ramp	the leading space clears the background to nothing

Settings that worked: **90 columns**, displayed at **460px**. Below about 88 columns the face muddies; much above it and the block dominates the page.

Rows are `cols * (h/w) * 0.48`, because monospace characters are about twice as tall as wide.

The typing animation

Each row sits in a `clipPath` whose rect animates `width` from 0 to full, with a small block riding the wipe edge as a cursor. Rows stagger top to bottom with `begin="{i * 0.09}s"`, and every animation uses `fill="freeze"` so the portrait prints **once** and stops. No looping.

The trap nobody mentions

The character grid bakes in an advance width of **exactly 0.600 em** — `CHAR_W = 7.74` at `font-size: 12.9`. Measured in a browser:

Font	Advance
Liberation Mono / DejaVu Sans Mono / Noto Sans Mono	0.600 
Ubuntu Mono	0.560
Consolas — what Windows lands on	≈ 0.55

 A Windows visitor sees your portrait about **7% narrower** than you do. Fix it by embedding the font — Part 4.

Part 2 — stats your own repo draws

Four graphics, all from the GitHub GraphQL API, all in the same visual language as the portrait:

- hero total + weekly sparkline
- current and longest streak, with date ranges
- top languages, by bytes and by repo
- the year at one character per day, using the portrait's own ramp

Pick the right chart type

The default activity graph is a line chart over daily counts — which is wrong, because daily contributions are sparse and discrete. A line through `0, 0, 11, 0, 0, 10` claims values that never existed. Columns are honest: a zero day is empty space. Save lines and areas for weekly aggregates, where continuity is defensible.

Two determinism traps

Both cost real time. Both produce a nightly stream of meaningless commits if you miss them.

1. Pin the window to whole UTC days. Left alone, `contributionsCollection` measures "the past year" from the moment of the request. Two runs minutes apart bucket days into different weeks and shift the sparkline by a fraction of a pixel — enough to look changed every night.

```
contributionsCollection(from: $from, to: $to) { ... }
```

with `from` = today – 364 days at `00:00:00Z`, and `to` = today at `23:59:59Z`.

2. Filter repositories to public only — `privacy: PUBLIC`. Your personal token sees private repos; the workflow's token doesn't. Without this, language percentages disagree depending on who ran the script.

The workflow

```
name: refresh stats on: schedule: - cron: "17 5 * * *" workflow_dispatch: # deliberately
no `push` — this job commits, and # a push trigger would re-run it on its own commit
permissions: contents: write jobs: refresh: runs-on: ubuntu-latest steps: - uses:
actions/checkout@v4 - env: GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }} GH_LOGIN: ${{
github.repository_owner }} run: python3 scripts/generate_stats.py - run: |
FILES="stats.svg streak.svg langs.svg year.svg hd-*.svg" [ -z "$(git status --porcelain
-- $FILES)" ] && exit 0 git config user.name "github-actions[bot]" git config user.email
"41898282+github-actions[bot]@users.noreply.github.com" git add -- $FILES && git commit
-m "stats: refresh" && git push
```

- **No personal access token needed.** The built-in `GITHUB_TOKEN` returns the same numbers.
- **Commit only on change**, or you get a commit every single night.
- **Let the action own the generated files.** Regenerating them locally too guarantees merge conflicts — your token and the workflow's bucket a day near a week boundary differently, so the output is never byte-identical.

The generator uses **only the Python standard library** — `urllib` for the API, no dependencies to break in CI.

Part 3 — making the text look deliberate

The graphics were finished and the body copy still looked like a default README. Given what GitHub strips, here's what's actually available.

Section headings as SVG. The only way to put your own typeface on a heading. Lowercase mono label with a hairline rule running to the right edge.

⚖ Stated plainly: image headings have no anchor links, so GitHub's README outline goes empty. The `alt` text carries the word for screen readers.

Control the line length. Full-width paragraphs run about 110 characters, which is a genuinely bad measure. Two options:

- A `width` attribute on a `td` works — but GitHub draws a border around table cells, so you get a visible box.
- Hard-wrap with `br` tags at about 76 characters. No border. Costs you clean reflow on narrow screens.

Prefer a samp tag over backticks. Both render monospace; `samp` does it *without* the boxed-chip background. Better for a stack row or per-project meta when you don't want twelve grey pills.

A blockquote lede. Gets a left rule and dimmer text for free — a real typographic device, no CSS required.

Part 4 — inlining the font

🔒 **An external font URL cannot work here.** These SVGs load through an `img` tag, and browsers refuse subresource fetches for image documents. A `@font-face` with a base64 data URI, however, works — verified by rendering the same file with and without it.

Which also means every SVG has to carry its own copy. Subset per role, or the page gets heavy.

Using JetBrains Mono — SIL OFL, and 600/1000 units, exactly the 0.600 the portrait grid assumes, so no geometry changes:

```
pip install fonttools brotli # just the 13 ramp characters, for the portrait pyftsubset
JetBrainsMono-Regular.ttf --text=' .`:-==*cs#%@\ \ --flavor=woff2 --layout-features='' -
-no-hinting -o ramp.woff2
```

Subset	Covers	Size
ramp	13 characters	1.3 KB
headings	only the letters used	1.4 KB
basic latin, 2 weights	the data graphics	4.5 KB each

About 57 KB across the whole page. Inlining a full TTF into each file instead would be roughly 4.5 MB.

Licence matters: the font file lands in a public repo, so it must be OFL or similar — JetBrains Mono, IBM Plex Mono, Fira Code, Source Code Pro. Ship the licence file next to it. Commercial fonts are not an option.

Gotchas, collected

- **A full-page screenshot restarts SMIL.** If you're verifying with headless Chrome, `fullPage: true` produces blank animated SVGs. Use a tall viewport instead, and wait — a 56-row portrait takes about 5.1 s to finish typing.
- **Pinned repositories and your bio cannot be set through the API.** No GraphQL mutation exists, and the REST call needs a `user` scope your CLI token won't have. Both are manual, in the UI.
- **Test markdown against the rendering API** — `POST /markdown` — before committing. It applies the same sanitiser as the site.
- **A newly created profile README is cached.** If it doesn't appear on your profile, edit it once through the web UI to force a refresh.
- **Don't colour per character.** Per-character rainbow colouring is what makes most ASCII portraits look like static. One fill colour.

Credits

- Portrait pipeline: ASCII Portrait README Guide — `burly-handstand-0dc.notion.site/ASCII-Portrait-README-Guide-3a3e3f86338481f0b545ec8120bbf604`
- Typeface: JetBrains Mono, SIL OFL 1.1
- All the code in this guide: andriidrok1/andriidrok1